

Agentic AI Pipeline for Coverage-Directed Unit Testing

CSE731: Software Testing — Mid-term Project, Term I (2026–27)

Krish Patel (IMT2023134) Yash Gupta (IMT2023125)

October 2026

Abstract

We built a pipeline of three agents, written in plain Java, that generates a unit of code, generates JUnit 5 unit tests that must reach a user-specified coverage criterion, and executes those tests to give a verdict. The test case generator follows option (1) of the problem statement: a coverage criterion (statement or branch coverage, measured with JaCoCo). The test case executor’s measured coverage gaps are fed back to the test case generator for up to three rounds. On the first 12 problems of HumanEval-X (Java), with a 100% branch-coverage target, 8 of 12 tasks reached the goal with all tests passing. Three of the remaining four contain branches that no valid input can reach.

Code repository: <https://github.com/kodercrish/Software-Testing>

1 Pipeline

1.1 Overview

The pipeline consists of three agents, connected by an ordinary Java loop (`Pipeline.java`). As the project rules require, it uses no agent framework, no RAG and no chain-of-thought: each LLM agent is a single HTTP request to an OpenAI-compatible chat API (OpenRouter).

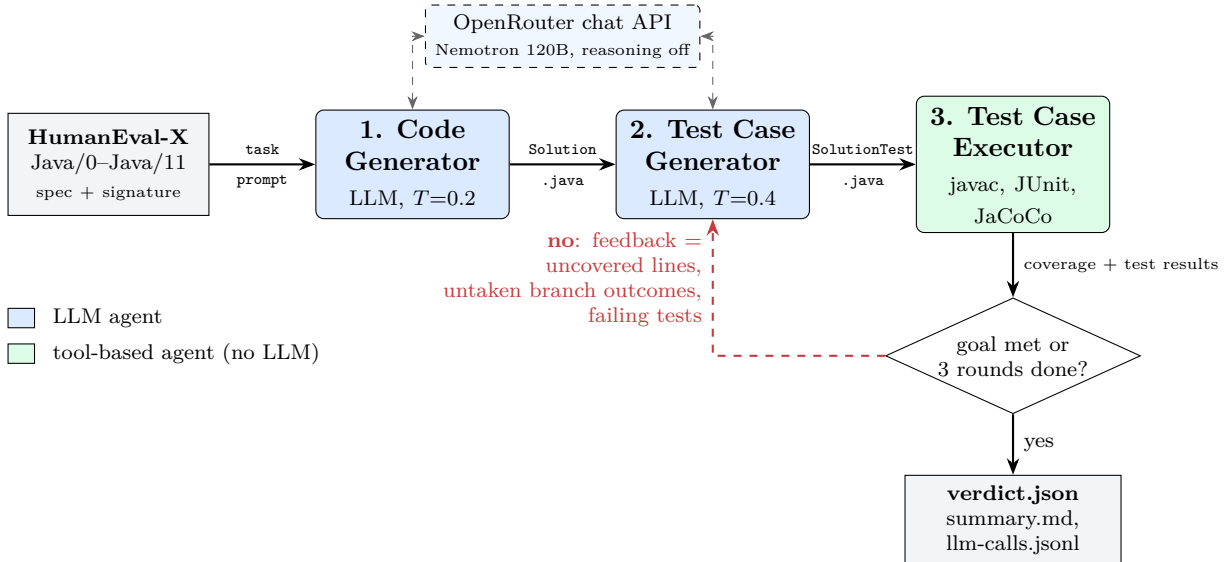


Figure 1: The agentic pipeline for one task. Agents 1 and 2 call the LLM; agent 3 runs real tools. If the coverage goal is not met, the executor’s findings go back to the test case generator (red, dashed).

1.2 Dataset

We use the Java split of **HumanEval-X** (164 problems), and run the pipeline on the first 12 (Java/0–Java/11). Each problem’s `prompt` contains the imports, `class Solution`, a Javadoc

specification with examples, and the method signature. Only this prompt is given to the agents.

1.3 Testing goal: user-specified coverage criterion

The user sets the criterion and the target in `config.properties`:

- **STATEMENT**: every executable statement is run by some test (JaCoCo *line* counter);
- **BRANCH**: every decision (`if`, loop condition, `&&`, `||`, `?:`) evaluates to both *true* and *false* (JaCoCo *branch* counter). This corresponds to edge coverage of the method’s control-flow graph.

For the experiments we used **BRANCH** with a target of 100%.

1.4 The three agents

1. **Code Generator (LLM)**. Receives the task prompt and returns one compilation unit, `Solution.java`. The Java code block is extracted from the reply.
2. **Test Case Generator (LLM)**. Receives the specification and the generated code *with line numbers*, and returns `SolutionTest.java` (JUnit 5). It is told to derive expected values from the *specification*, not from the implementation, so that tests can reveal bugs. In a feedback round it also receives the executor’s report and the current tests, and returns an improved test class.
3. **Test Case Executor (no LLM)**. Compiles and runs the tests in a separate JVM, measures coverage and decides the verdict (Figure 2). Because the verdict comes from real execution, it is objective and reproducible.

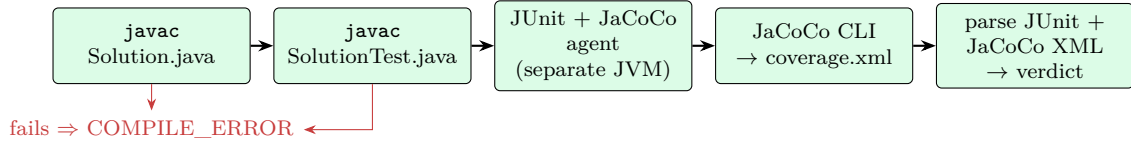


Figure 2: Steps of the Test Case Executor. Every process has a 30s timeout.

The verdict is decided by four rules, checked in order:

Condition	Verdict
code or tests do not compile	COMPILE_ERROR
no test ran, or any test failed	TESTS_FAILED
all tests pass, coverage < target	COVERAGE_NOT_MET
all tests pass, coverage $\geq$ target	PASS

1.5 Feedback loop

If the goal is not met, the executor’s results are turned into a plain-text report and sent back to the test case generator. The report lists the uncovered lines with their source text, the decisions with untaken outcomes, failing tests with their messages, and compile errors. The loop stops when the goal is met, when the generated code itself does not compile, or after `maxTestIterations` = 3 rounds. The verdict of the last round is reported.

2 Prompts and Settings

Setting	Code Generator	Test Case Generator
Model	nvidia/nemotron-3-super-120b-a12b:free via OpenRouter	
Reasoning (“thinking”)	disabled	disabled
temperature	0.2	0.4
top_p	0.95	0.95
max_tokens	2048	4096
seed	42	42
Other	—	criterion = BRANCH, target = 100%, max. 3 rounds

Table 1: Model and sampling settings.

- The code generator uses a low temperature to get the most likely correct solution.
- The test generator uses a slightly higher temperature, so that it tries more varied inputs when it must reach uncovered branches.
- The model’s built-in reasoning mode is switched off in every request (`"reasoning": {"enabled": false}`), so it answers directly without a hidden chain of thought.
- The free model is sometimes rate-limited, so the client retries a failed request after 20 s, up to 6 times.

The prompts are plain-text templates. Placeholders `{{...}}` are filled in at run time, and every request and response is logged in `llm-calls.jsonl`.

Code Generator, system prompt

```
You are a code generator agent in an automated unit-testing pipeline.
You write correct, compilable Java 21 code that implements a specification.

Rules:
- Output exactly one Java code block (```java ... ```) and nothing else.
- The code block must be a complete compilation unit: the given imports and the class `Solution`
  containing the requested method with the exact given signature. Helper methods are allowed inside `Solution`.
- Do not add a package declaration, a main method, or any other top-level class.
- Use only the Java standard library.
```

Code Generator, user prompt

```
Implement the method described by the Javadoc below. Keep the class name, method name, parameter types and
return type exactly as given.

```java
{{prompt}}
```
```

Test Case Generator, system prompt

```
You are a test case generator agent in an automated unit-testing pipeline.
You write JUnit 5 unit tests for a single Java method.

Testing goal: {{criterion}}. Target: {{target}}% as measured by JaCoCo.

Rules:
- Output exactly one Java code block (```java ... ```) and nothing else.
- The code block is a complete compilation unit containing the class `SolutionTest` (no package declaration).
```

- Import `org.junit.jupiter.api.Test` and static `org.junit.jupiter.api.Assertions.*`, plus any `java.util` classes you use.
- Create the object under test with `new Solution()`.
- Choose inputs so that the testing goal is reached: every statement / every decision outcome of the given implementation must be exercised by at least one test.
- Derive every expected value from the SPECIFICATION (the Javadoc and its examples), not from the implementation, so that a test fails if the implementation is wrong.
- One scenario per `@Test` method, with a descriptive method name. Use `assertEquals` / `assertTrue` / `assertFalse`; for doubles use `assertEquals(expected, actual, 1e-6)`.
- Do not use mocking libraries, parameterized tests, reflection, randomness, or file / network I/O.

Test Case Generator, user prompt

Specification of the method under test:

```
```java
{{prompt}}
```
```

Implementation under test (Solution.java, with line numbers):

```
```
{{numberedCode}}
```
```

Write the JUnit 5 class `SolutionTest`.

Test Case Generator, feedback prompt (appended to the user prompt in rounds 2 and 3)

The Test Executor ran your tests. Results:

```
{{feedback}}
```

Current `SolutionTest.java`:

```
```java
{{tests}}
```
```

Return the complete improved `SolutionTest` class in one Java code block.

- Add tests that exercise the uncovered lines / branch outcomes listed above.
- If a failing test's expected value contradicts the specification, fix that test. If it agrees with the specification, keep it unchanged: it may have found a bug in the implementation.
- Keep all existing passing tests.

3 Formats of Code, Tests and Verdict

- **Code:** one Java file `Solution.java` containing class `Solution` with the required method.
- **Tests:** one JUnit 5 file `SolutionTest.java` with one scenario per `@Test` method.
- **Verdict:** `verdict.json`, written by the test case executor.

As a sample we show task Java/11 (`stringXor`), which needed one feedback round.

Generated code (`Solution.java`)

```
import java.util.*;
import java.lang.*;

class Solution {
    /**
```

Input are two strings a and b consisting only of 1s and 0s.
Perform binary XOR on these inputs and return result also as a string.

```
>>> stringXor("010", "110")
```

```
"100"
```

```
*/
```

```
public String stringXor(String a, String b) {  
    StringBuilder result = new StringBuilder();  
    int length = Math.max(a.length(), b.length());  
    for (int i = 0; i < length; i++) {  
        char bitA = i < a.length() ? a.charAt(a.length() - 1 - i) : '0';  
        char bitB = i < b.length() ? b.charAt(b.length() - 1 - i) : '0';  
        char xorBit = (bitA == bitB) ? '0' : '1';  
        result.insert(0, xorBit);  
    }  
    return result.toString();  
}  
}
```

Generated tests, final round (SolutionTest.java)

```
import org.junit.jupiter.api.Test;  
import static org.junit.jupiter.api.Assertions.*;  
  
class SolutionTest {  
    @Test  
    void testEqualLengthBothZeros() {  
        Solution sol = new Solution();  
        assertEquals("000", sol.stringXor("000", "000"));  
    }  
  
    @Test  
    void testEqualLengthBothOnes() {  
        Solution sol = new Solution();  
        assertEquals("000", sol.stringXor("111", "111"));  
    }  
  
    @Test  
    void testEqualLengthMixed() {  
        Solution sol = new Solution();  
        assertEquals("110", sol.stringXor("101", "011"));  
    }  
  
    @Test  
    void testFirstLonger() {  
        Solution sol = new Solution();  
        assertEquals("1001", sol.stringXor("1100", "101"));  
    }  
  
    @Test  
    void testSecondLonger() {  
        Solution sol = new Solution();  
        assertEquals("1001", sol.stringXor("101", "1100"));  
    }  
  
    @Test  
    void testOneEmpty() {  
        Solution sol = new Solution();  
        assertEquals("101", sol.stringXor("", "101"));  
    }  
  
    @Test  
    void testBothEmpty() {  
        Solution sol = new Solution();  
    }  
}
```

```

    assertEquals("", sol.stringXor("", ""));
}

@Test
void testSingleCharDifferent() {
    Solution sol = new Solution();
    assertEquals("1", sol.stringXor("0", "1"));
}

@Test
void testSingleCharSame() {
    Solution sol = new Solution();
    assertEquals("0", sol.stringXor("1", "1"));
}
}

```

Verdict (verdict.json)

```

{
  "taskId": "Java/11",
  "status": "PASS",
  "testsTotal": 9,
  "testsPassed": 9,
  "criterion": "BRANCH",
  "target": 100.0,
  "statementPct": 100.0,
  "branchPct": 100.0,
  "iterations": 2
}

```

4 Execution Results

The run on 4 October 2026 used tasks Java/0–Java/11, criterion **BRANCH**, a 100% target and at most 3 rounds. All artefacts are in `runs/20261004-182338/`.

| Task | Method | Verdict | Tests passed | Stmt. % | Branch % | Rounds |
|--|-----------------------|------------------|--------------|---------|----------|--------|
| Java/0 | hasCloseElements | COMPILE_ERROR | 0/0 | 0.0 | 0.0 | 3 |
| Java/1 | separateParenGroups | TESTS_FAILED | 13/16 | 100.0 | 83.3 | 3 |
| Java/2 | truncateNumber | PASS | 4/4 | 100.0 | 100.0 | 1 |
| Java/3 | belowZero | PASS | 6/6 | 100.0 | 100.0 | 1 |
| Java/4 | meanAbsoluteDeviation | PASS | 7/7 | 100.0 | 100.0 | 1 |
| Java/5 | intersperse | PASS | 5/5 | 100.0 | 100.0 | 1 |
| Java/6 | parseNestedParens | COVERAGE_NOT_MET | 9/9 | 100.0 | 93.8 | 3 |
| Java/7 | filterBySubstring | PASS | 5/5 | 100.0 | 100.0 | 1 |
| Java/8 | sumProduct | PASS | 5/5 | 100.0 | 100.0 | 1 |
| Java/9 | rollingMax | PASS | 8/8 | 100.0 | 100.0 | 1 |
| Java/10 | makePalindrome | TESTS_FAILED | 10/11 | 100.0 | 91.7 | 3 |
| Java/11 | stringXor | PASS | 9/9 | 100.0 | 100.0 | 2 |
| Coverage goal reached with all tests passing: 8/12 tasks | | | | | | |

Table 2: Final verdict per task (last round).

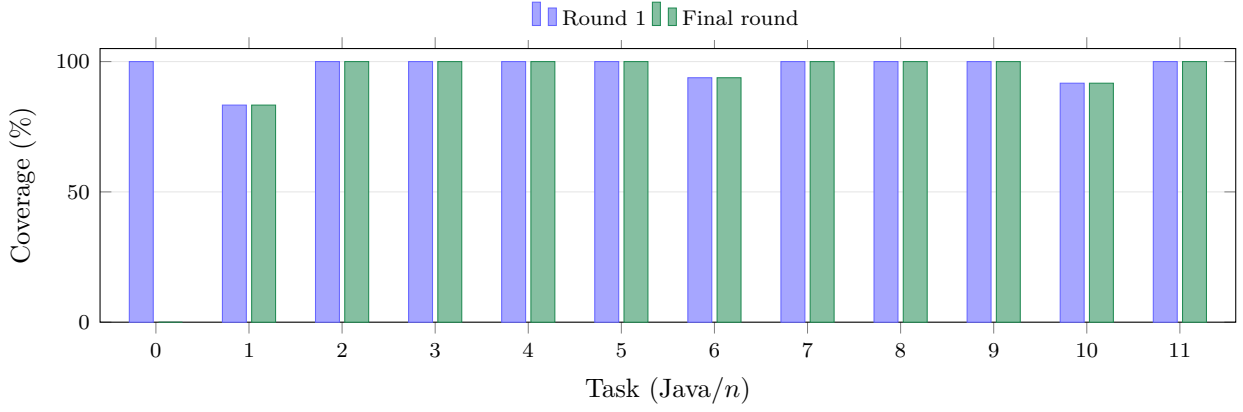


Figure 3: Branch coverage per task in round 1 and in the final round (target 100%). For Java/0, the final round did not compile.

| Task | Round 1 | Round 2 | Round 3 |
|---------|-------------------|-------------------|-------------------|
| Java/0 | 9/10 pass, 100.0% | 9/13 pass, 100.0% | does not compile |
| Java/1 | 8/8 pass, 83.3% | 10/12 pass, 83.3% | 13/16 pass, 83.3% |
| Java/6 | 7/7 pass, 93.8% | 8/8 pass, 93.8% | 9/9 pass, 93.8% |
| Java/10 | 7/7 pass, 91.7% | 9/9 pass, 91.7% | 10/11 pass, 91.7% |
| Java/11 | 7/9 pass, 100.0% | 9/9 pass, 100.0% | — |

Table 3: Tasks that used the feedback loop: tests passed and branch coverage per round. All other tasks passed in round 1.

Feedback sent to the test case generator after round 1 of **Java/11**:

```
Coverage: statements 9/9 (100.0%), branches 8/8 (100.0%)
Tests: 9 run, 7 passed, 2 failed
  FAILED testFirstLonger(): expected: <1010> but was: <1001>
  FAILED testSecondLonger(): expected: <1010> but was: <1001>
```

4.1 Discussion

- **Goal reached in 8 of 12 tasks**, 7 of them in the first round. All tasks whose final round compiled reached 100% statement coverage.
- **The feedback loop repaired wrong tests.** In Java/11 two tests expected wrong XOR results (1010 instead of 1001). After the executor reported the failures, the generator corrected them in round 2.
- **Infeasible branches (Java/1, Java/6, Java/10).** The uncovered branch outcomes cannot be reached by any input that satisfies the specification:
 - In Java/1 and Java/6, the *false* outcome of `else if (c == '')` needs a character that is neither (nor), but the input contains only parentheses and spaces.
 - In Java/10, the exit of the `for` loop is never reached, because a single character is always a palindromic suffix, so the loop always returns early.

These are infeasible test requirements: 100% branch coverage is not achievable for these implementations. Trying anyway, the generator invented invalid inputs (e.g. unbalanced parentheses) with guessed expected values, which then failed. An LLM cannot tell an infeasible branch from a hard one.

- **Wrong oracles caught by execution.** Some generated expected values contradict the specification. For example, Java/0 expects `true` for threshold 0, although no difference is

smaller than 0, and Java/10 expects `makePalindrome("ab")` to be "ababa" instead of "aba". Such errors are found only because the verdict comes from running the tests, not from LLM judgement.

- **Regression across rounds (Java/0).** Round 1 already reached 100% coverage with one wrong test. In round 3 the generator produced duplicate test-method names, so the suite no longer compiled. Since the pipeline reports the last round, the verdict is `COMPILE_ERROR`. Keeping the best round would be a simple improvement.

5 Contributions

The work was split into the LLM side of the pipeline and the execution side.

| Member | Contribution |
|--------------------------|---|
| Yash Gupta (IMT2023125) | Part 1: LLM agents and orchestration. OpenRouter client (<code>LlmClient</code>), Code Generator agent, Test Case Generator agent including the feedback message, prompt templates and sampling settings (<code>Prompts</code> , <code>config.properties</code>), dataset loader (<code>HumanEvalX</code>), and the pipeline loop (<code>Pipeline</code> , <code>Main</code>). |
| Krish Patel (IMT2023134) | Part 2: Test execution, results and report. Test Case Executor agent (compiling and running tests with JUnit and JaCoCo, verdict), process runner (<code>Proc</code>), JUnit/JaCoCo report parsing (<code>Reports</code>), run logging and results summary (<code>RunLogger</code>), running the experiments on the 12 tasks, and writing this report. |